

Internet of Things (IoT) / Internet des Objets (IdO)

Application Protocols : MQTT

Gilles Menez
University Nice Côte d'Azur
email : menez@unice.fr
www : www.i3s.unice.fr/~menez

November 19, 2025: V 1.0

Table of Contents

1	Mosquitto	5
1.1	Installation d'un broker en local	5
1.2	Ca tourne ?	5
1.3	Configuration	6
1.4	Test fonctionnel du serveur/broker	7
1.5	Dump du serveur	8
1.6	Un serveur dans le Cloud/public ?	9
2	API MQTT pour ESP32	10
2.1	Installation de l'API "pubsubclient"	10
3	Expérimentation : Un premier exemple	11
3.1	MQTT pour l'ESP32	11
3.1.1	Analyse du code	15
3.1.2	Just for information : An example of asynchronous MQTT	16
3.2	Others MQTT client APIs	17
3.2.1	With mosquitto CLI	17
3.2.2	With a client in a(ny) programming language (optionnal)	17
3.2.3	With a client in Python	17
3.2.4	Few comments on this code	19
3.2.5	With a client in Java (optional work !)	19
4	TODO 1 : La connexion avec Node-Red	20
4.1	Au niveau de l'ESP	20
4.2	Au niveau de Node Red	20
5	Consommation (Optionnel !)	21

List of Figures

What you should learn ...

1. We use another application protocol : MQTT
2. The paradigm is different !

1 Mosquitto

1.1 Installation d'un broker en local

On pourrait n'installer qu'un client MQTT sur votre machine et utiliser un des nombreux brokers disponibles sur Internet.

Mais pour réduire votre dépendance à la disponibilité d'un réseau Internet et comme c'est assez simple à faire, vous pouvez installer client et serveur/broker sur votre machine.

<https://mosquitto.org/download/>

- Vous pourriez aussi passer par un container ... moi j'apt-get ;-)

Du coup, on installe les deux :

➤ Le broker :

```
sudo apt-get install mosquitto
```

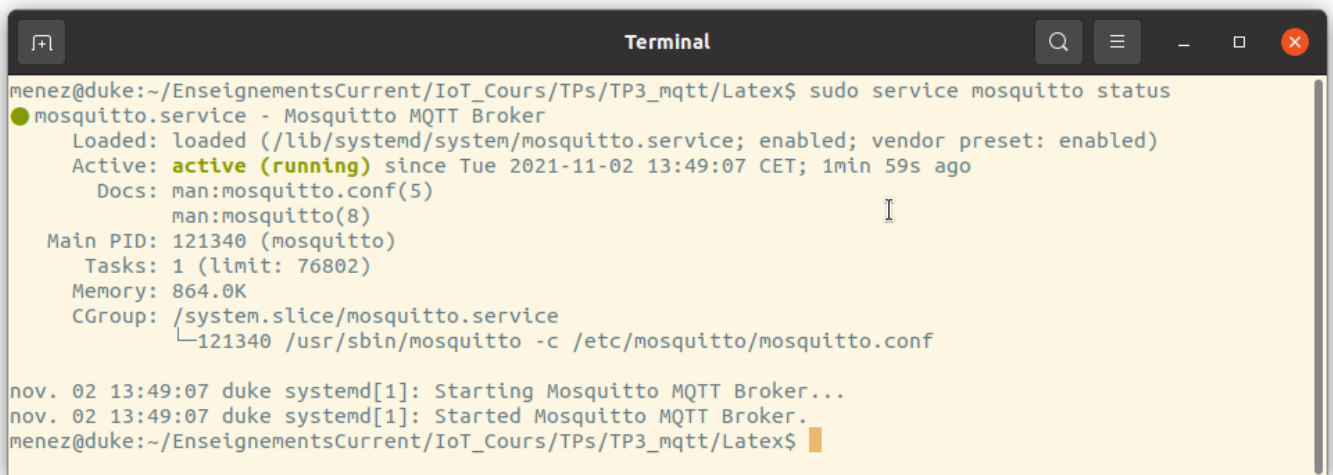
➤ La CLI (mosquitto_pub, mosquitto_sub, mosquitto_passwd ...) qui permet de faire de votre machine un client d'un broker local ou distant :

```
sudo apt-get install mosquitto-clients
```

1.2 Ca tourne ?

Sous Linux, vous pouvez vérifier que le service/broker "tourne" :

```
sudo service mosquitto status
```



```
Terminal
menez@duke:~/EnseignementsCurrent/IoT_Cours/TPs/TP3_mqtt/Latex$ sudo service mosquitto status
● mosquitto.service - Mosquitto MQTT Broker
   Loaded: loaded (/lib/systemd/system/mosquitto.service; enabled; vendor preset: enabled)
   Active: active (running) since Tue 2021-11-02 13:49:07 CET; 1min 59s ago
     Docs: man:mosquitto.conf(5)
           man:mosquitto(8)
   Main PID: 121340 (mosquitto)
     Tasks: 1 (limit: 76802)
    Memory: 864.0K
    CGroup: /system.slice/mosquitto.service
            └─121340 /usr/sbin/mosquitto -c /etc/mosquitto/mosquitto.conf

nov. 02 13:49:07 duke systemd[1]: Starting Mosquitto MQTT Broker...
nov. 02 13:49:07 duke systemd[1]: Started Mosquitto MQTT Broker.
menez@duke:~/EnseignementsCurrent/IoT_Cours/TPs/TP3_mqtt/Latex$
```

et si il le faut, éventuellement manipuler le service (post reconfiguration ?) :

```
sudo service mosquitto start
sudo service mosquitto stop
sudo service mosquitto restart
```

Vous remarquez sur la console que le serveur MQTT s'exécute "avec" le fichier de configuration:

`/etc/mosquitto/mosquitto.conf`

1.3 Configuration

```
1  #File : /etc/mosquitto/mosquitto.conf
2  # A full description of the configuration file is at
3  # /usr/share/doc/mosquitto/examples/mosquitto.conf.example
4
5  #A PID file is just a regular file on disk that contains the PID
6  #(Process IDentifier) number of the primary process of a system
7  #service. System service managers such as systemd and OpenRC use this
8  #number to manage the health of the service while it's running and send
9  #it various signals (such as to ask it to reload its configuration
10 #file).
11 pid_file /run/mosquitto/mosquitto.pid
12
13 persistence true
14 persistence_location /var/lib/mosquitto/
15
16 log_dest file /var/log/mosquitto/mosquitto.log
17
18 # Place your local configuration in /etc/mosquitto/conf.d/
19 include_dir /etc/mosquitto/conf.d
```

Ce fichier

- active la persistance, c'est à dire la sauvegarde des données reçues / envoyées en MQTT.
On précise le répertoire ("/var/lib/mosquitto") contenant le "persistence_file" ("mosquitto.db")
- puis "appelle" (include) le fichier personnalisé (par moi au plus simple) suivant :

```
1  #File : /etc/mosquitto/conf.d/gm.conf
2  # Changement => sudo service mosquitto restart
3  #the default config will only bind to localhost as a move to
4  #a more secure default posture compared to previous MQTT versions.
5
6  #Section Extra Listener :
7  #Running the broker with a listener defined will bind by default to
8  #0.0.0.0 / :: and so will be accessible from any interface.
9  #It is still possible to bind to a specific address/interface !.
10 listener 1883
11 protocol mqtt
12
13 #Section Connection :
14 #By default it will also only allow anonymous connections (without
15 #username/password) from localhost, to allow anonymous from REMOTE
16 #add:
17 allow_anonymous true
```

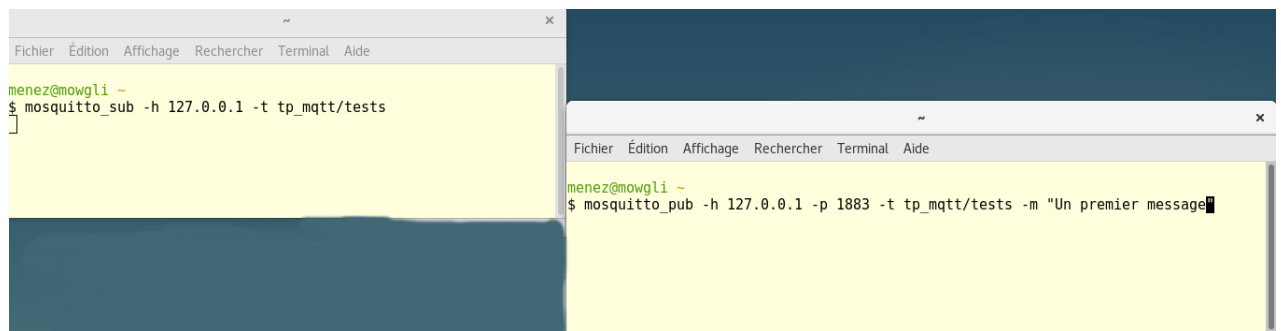
Quelques remarques :

- La ligne listener 1883 crée un nouveau "listener" qui va écouter sur le port 1883 pour toutes les interfaces réseaux (et non pas seulement sur l'hôte local (localhost) comme par défaut)
- On indique avec le paramètre protocol qu'il s'agit de mqtt.
- On est loin d'être "secure" avec ce type de configuration !
On s'essayera de s'occuper de ce problème plus tard ... dans une autre vie, vu le volume horaire accordé :-)

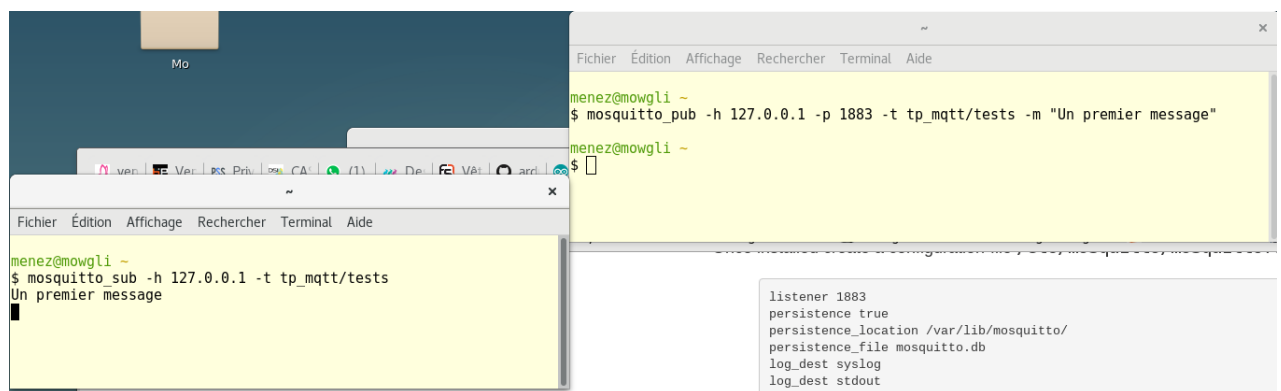
1.4 Test fonctionnel du serveur/broker

On utilise le serveur que vous venez d'installer sur votre machine (127.0.0.1) :

- ① Un client souscrit à un topic "tp_mqtt/tests"



- ② Un autre publie un message sur ce même topic.
- ③ Le broker (sur le localhost) fait immédiatement suivre



Les options de verbose (-v) et debug (-d) peuvent apporter des informations ... en cas de problème ... cela va venir !

1.5 Dump du serveur

Avec la commande qui suit, vous aurez un dump des méta-informations contenues dans le serveur (broker) :

```
mosquitto_sub -h localhost -v -t \${SYS}/#
```

- `\${SYS}/broker/load/bytes/received` :
The total number of bytes received since the broker started.
- `\${SYS}/broker/load/bytes/sent` :
The total number of bytes sent since the broker started.
- `\${SYS}/broker/clients/connected` :
The number of currently connected clients
- `\${SYS}/broker/clients/disconnected` :
The total number of persistent clients (with clean session disabled) that are registered at the broker but are currently disconnected.
- `\${SYS}/broker/clients/maximum` :
The maximum number of active clients that have been connected to the broker. This is only calculated when the `\${SYS}` topic tree is updated, so short lived client connections may not be counted.
- `\${SYS}/broker/clients/total` :
The total number of connected and disconnected clients with a persistent session currently connected and registered on the broker.
- `\${SYS}/broker/messages/received` :
The total number of messages of any type received since the broker started.
- `\${SYS}/broker/messages/sent` :
The total number of messages of any type sent since the broker started.
- `\${SYS}/broker/messages/publish/dropped` :
The total number of publish messages that have been dropped due to inflight/queuing limits.
- `\${SYS}/broker/messages/publish/received` :
The total number of PUBLISH messages received since the broker started.
- `\${SYS}/broker/messages/publish/sent` :
The total number of PUBLISH messages sent since the broker started.
- `\${SYS}/broker/messages/retained/count` :
The total number of retained messages active on the broker.
- `\${SYS}/broker/subscriptions/count` :
The total number of subscriptions active on the broker.
- `\${SYS}/broker/time` :
The current time on the server.
- `\${SYS}/broker/uptime` :
The amount of time in seconds the broker has been online.
- `\${SYS}/broker/version` :
The version of the broker. Static.

1.6 Un serveur dans le Cloud/public ?

Si vous voulez utiliser un serveur/broker visible sur Internet (dans le cloud), il y a des solutions commerciales et aussi gratuites (parfois avec des restrictions).

On mentionnera tout particulièrement :

- <https://mosquitto.org/>, avec son broker

`test.mosquitto.org`

- <https://www.hivemq.com/mqtt-demo/>, avec son broker

`broker.hivemq.com`

"Our public HiveMQ MQTT broker is open for anyone to use. Feel free to write an MQTT client that connects with this broker. We have a dashboard so you can see the amount of traffic on this broker. We also keep a list of MQTT client libraries that can be used to connect to HiveMQ."

- <https://iot.eclipse.org/getting-started/>, avec son broker

`iot.eclipse.org`

- <https://docs.shiftr.io/interfaces/mqtt/>, avec son broker

`broker.shiftr.io`

2 API MQTT pour ESP32

Pour que l'ESP puisse manipuler MQTT il y a plusieurs possibilités parmi lesquelles :

1. L'API d'Espressif (les concepteurs de l'ESP).

<https://github.com/espressif/esp-mqtt>

2. Les bibliothèques de l'IDE Arduino, et tout particulièrement la bibliothèque : `"pubsubclient"` (by O'Leary) qui propose une approche **synchrone** :

<https://github.com/knolleary/pubsubclient>

Il faut noter que d'autres API clients MQTT existent parmi lesquelles :

- <https://github.com/256dpi/arduino-mqtt>
- <https://www.esp8266.com/viewtopic.php?t=8172>
- <https://github.com/marvinroger/async-mqtt-client>

qui propose une **bibliothèque asynchrone** !

Pas forcément simple d'avoir plusieurs "serveurs asynchrones" (HTTP et MQTT) en même temps sur un ESP ?

2.1 Installation de l'API "pubsubclient"

Pour l'installer à partir de l'IDE Arduino :

A partir du menu "Croquis/Inclure une bibliothèque/Gérer les bibliothèques"

chercher **"pubsubclient"**.

La **documentation** est plutôt bien :

- <https://pubsubclient.knolleary.net/api.html>
- <http://www.steves-internet-guide.com/using-arduino-pubsub-mqtt-client/>

ATTENTION de bien lire les limitations ...notamment sur la taille d'un message standard !!!

- <https://github.com/knolleary/pubsubclient/issues/501>

Si votre payload est plus grande, il faudra le spécifier !

Par exemple, pour 512 octets (quelle taille environ votre Json ?) : `client.setBufferSize(512);`

Pour les MacOS ...merci Robin :

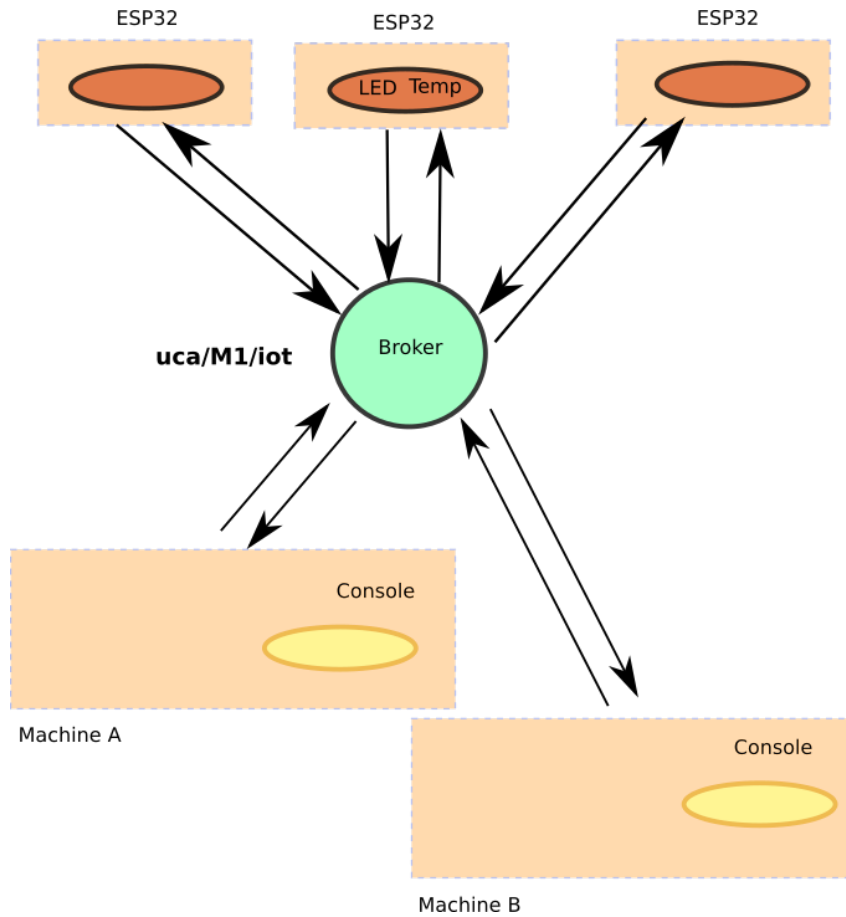
Pour le problème sur MacOS du PubSubClient.h introuvable il faut l'installer manuellement :

1. Télécharger <https://github.com/knolleary/pubsubclient/releases/tag/v2.7>
2. `unzip ~/Downloads/pubsubclient-2.7.zip`
3. `mv ~/Downloads/pubsubclient-2.7 ~/Documents/Arduino/libraries`

3 Expérimentation : Un premier exemple

Pour appréhender MQTT, vous allez expérimenter une utilisation simple.

L'exemple qui suit **et que je vous demande d'étudier** montre comment un ESP peut utiliser MQTT pour **diffuser** ses résultats et aussi **recevoir** des informations :



En utilisant le broker "test.mosquitto.org",

1. L'ESP publie
 - sa température sur le topic : "**uca/M1/iot/temp**"
 - et l'état de sa LED (Pin 2) sur le topic et "**uca/M1/iot/led**"
2. L'ESP veut aussi recevoir, via ce broker sur les mêmes topics, des informations ... par exemple les températures des "autres" ?

3.1 MQTT pour l'ESP32

Ce code est **FOURNI SUR LE SITE** ! ...vous devez le comprendre et utiliser pour cela la documentation de l'API "pubsubclient" :

```

1  /*****
2      Based on Rui Santos work : https://randommerdtutorials.com/esp32-mqtt-publish-subscribe-arduino-ide/
3      File mqtt_full/mqtt_full.ino
4      Modified by GM
5
6      Tests with CLI :
7
8      mosquitto_pub -h test.mosquitto.org -t "uca/M1/iot/led" -m "{\"led\" : \"ON\"}" -q 1
9      mosquitto_sub -h test.mosquitto.org -t "uca/M1/iot/temp" -q 1
10  *****/
11  #include <WiFi.h>
12  #include <PubSubClient.h>
13  #include <Wire.h>
14  #include "wifi_utils.h"
15  #include "OneWire.h"
16  #include "DallasTemperature.h"
17
18  /*===== GPIO =====*/
19  const int ledPin = 2; // LED Pin
20  OneWire oneWire(23); // TEMP Pin
21  DallasTemperature tempSensor(&oneWire) ; // via OneWire
22
23
24  #define MQTT_HOST IPAddress(192, 168, 1, XXX)
25
26  /*===== MQTT broker/server =====*/
27  //const char* mqtt_server = "192.168.1.101";
28  //const char* mqtt_server = "public.cloud.shiftr.io"; // Failed in 2021
29  // need login and passwd (public,public) mqtt://public:public@public.cloud.shiftr.io
30  //const char* mqtt_server = "broker.hivemq.com"; // anonymous Ok in 2021
31  const char* mqtt_server = "test.mosquitto.org"; // anonymous Ok in 2021
32  //const char* mqtt_server = "mqtt.eclipseprojects.io"; // anonymous Ok in 2021
33  /*===== MQTT TOPICS =====*/
34  #define TOPIC_TEMP "uca/M1/iot/temp"
35  #define TOPIC_LED "uca/M1/iot/led"
36  /*===== ESP is a MQTT Client =====*/
37  WiFiClient espClient; // Wifi
38  PubSubClient mqttclient(espClient); // MQTT client
39  String hostname = "Mon petit objet ESP32";
40
41  #define USE_SERIAL Serial
42
43  /*===== Arduino IDE paradigm : setup+loop =====*/
44  void setup() {
45      // Serial port
46      Serial.begin(9600);
47      while (!Serial); // wait for a serial connection. Needed for native USB port only
48
49      // GPIOs configuration

```

```

50     pinMode(ledPin, OUTPUT);
51     set_LED(HIGH); // Set led to HIGH
52
53     tempSensor.begin(); // Init temperature sensor
54
55     // Wifi connection
56     wifi_connect_multi(hostname);
57     wifi_printstatus(0);
58
59     // set server of our MQTT client
60     mqttclient.setServer(mqtt_server, 1883);
61     // set callback when publishes arrive for the subscribed topics
62     mqttclient.setCallback(mqtt_pubcallback);
63 }
64
65 /*===== TO COMPLETE ? =====*/
66 void set_LED(int v){
67     digitalWrite(ledPin, v);
68 }
69
70 float get_Temperature(){
71     return 22.5;
72 }
73
74 /*===== CALLBACK =====*/
75 void mqtt_pubcallback(char* topic,
76                       byte* payload,
77                       unsigned int length) {
78     /*
79      * Callback when a message is published on a subscribed topic.
80      */
81     USE_SERIAL.print("Message arrived on topic : ");
82     USE_SERIAL.println(topic);
83     USE_SERIAL.print("=> ");
84
85     // Byte list (of the payload) to String and print to Serial
86     String message;
87     for (int i = 0; i < length; i++) {
88         //USE_SERIAL.print((char)payload[i]);
89         message += (char)payload[i];
90     }
91     USE_SERIAL.println(message);
92
93     /*
94     char msg[length + 1];
95     memcpy(msg, payload, length);
96     msg[length] = NULL;
97     message = String(msg);
98     */

```

```

99
100 // Feel free to add more if statements to control more GPIOs with MQTT
101
102 // If a message is received on the topic, you check if the message is either "on" or "off".
103 // Here we change the output state according to the message ... why not !
104 if (String(topic) == TOPIC_LED) {
105     USE_SERIAL.print("so ... changing output to ");
106     if (message == "on") {
107         USE_SERIAL.println("on");
108         set_LED(HIGH);
109     }
110     else if (message == "off") {
111         USE_SERIAL.println("off");
112         set_LED(LOW);
113     }
114 }
115 }
116
117 /*===== SUBSCRIBE to TOPICS =====*/
118 void mqtt_subscribe_mytopics() {
119     /*
120     * Subscribe to MQTT topics :
121     * There is no way on checking the subscriptions from a client.
122     * But you can also subscribe WHENEVER you connect.
123     * Then it is guaranteed that all subscriptions are existing.
124     * => If the client is already connected then we have already subscribe
125     * since connection and subscriptions go together !
126     */
127     // Checks whether the client is connected to the MQTT server
128     while (!mqttclient.connected()) { // Loop until we're reconnected https://pubsubclient.knolleary.net/api
129         USE_SERIAL.print("Attempting MQTT connection...");
130
131         // Create a client ID from MAC address .. should be unique ascii string and different from all other devices usi
132         String mqttclientId = "ESP32-";
133         mqttclientId += WiFi.macAddress(); // if we need random : String(random(0xffff), HEX);
134         if (mqttclient.connect(mqttclientId.c_str(), // Mqttclient Id when connecting to the server : 8-12 alphanumeric
135                               NULL, /* No credential */
136                               NULL))
137         {
138             USE_SERIAL.println("connected");
139
140             // THEN Subscribe topics
141             mqttclient.subscribe(TOPIC_LED,1);
142             // mqttclient.subscribe(another topic ?);
143         }
144         else { // Connection to broker failed : retry !
145             USE_SERIAL.print("failed, rc=");
146             USE_SERIAL.print(mqttclient.state());
147             USE_SERIAL.println(" try again in 5 seconds");

```

```

148     delay(5000); // Wait 5 seconds before retrying
149 }
150 } // end while
151 }
152
153 /*===== LOOP =====*/
154 void loop() { /*--- Publish Temperature periodically */
155     int32_t period = 5000; // 5 sec
156     float temperature;
157
158     /*--- subscribe to TOPIC_LED if not yet ! */
159     mqtt_subscribe_mytopics();
160
161     delay(period);
162
163     /*--- make payload ... too badly ! */
164     temperature = get_Temperature();
165
166     char payload[100];
167     sprintf(payload,
168         "{\"temperature\" : \"%.2f\", \"by\" : \"%s\" }", temperature, WiFi.macAddress().c_str()); // Convert the va
169
170     // Serial info
171     USE_SERIAL.print("Published payload : "); USE_SERIAL.print(payload);
172     USE_SERIAL.print(" on topic : "); USE_SERIAL.println(TOPIC_TEMP);
173
174     /*--- Publish payload on TOPIC_TEMP */
175     mqttclient.publish(TOPIC_TEMP, payload);
176
177     /* Process MQTT ... une fois par loop() ! */
178     mqttclient.loop(); // Process MQTT event/action
179 }
180

```

3.1.1 Analyse du code

- ① Une phase préparatoire au dialogue et aux échanges.

Ligne 31, on déclare le broker et on définit les topics avec notamment le topic de température à la ligne 34.

On choisit d'utiliser un topic par capteur ... pas certain de l'optimalité de cette solution ? Mais pour l'instant on se focalise sur les topics et leur gestion.

A la ligne 38, l'ESP est un "client" WiFi et ce lien réseau sera utilisé par le client MQTT.

- ② Ligne 60, on associe (mais **sans le connecter**) le broker à notre ESP (qui est client) et on choisit le port 1883 (donc classique non sécurisé)
- ③ Ligne 62 : on associe aussi un "callback" au client qui sera invoqué à la réception d'une publication sur un des topics auxquels l'ESP pourrait avoir souscrits.

C'est le broker qui fait suivre cette publication.

- ④ Dans ce callback (Ligne 75), on analyse le message MQTT reçu pour en déduire le topic concerné.

On peut avoir souscrit à plusieurs topics !

- ⑤ La connexion au broker est réalisée par la fonction `mqtt_subscribe_mytopics` à la ligne 118.

Cette fonction permet de plus de souscrire aux topics qui nous intéresse : Ligne 141

Je ne fais que la moitié du travail puisque je ne souscris qu'à "led"

Pourquoi coupler les deux (connexion et topic) ?

Deux raisons, la première est que le client n'a pas de moyen de savoir si il a déjà souscrit et la seconde raison est que si il y a eu une déconnexion du réseau physique, le comportement du broker n'est pas garanti et rien n'assure que la souscription a persisté.

En liant les deux on peut savoir que si on est connecté alors on a forcément souscrit !

Dans cet exemple, l'utilisation de la fonction "connect" (Ligne 134) est **simpliste** :

- ✓ un identifiant construit l'adresse MAC de l'ESP ;
Le premier paramètre ("esp32") est l'identifiant (clientId) du client (l'ESP) au niveau du broker.
Ce n'est pas le "UserId" (qui fait partie des credentials) dont on verra plus loin qu'il peut être utilisé pour contrôler les connexions au broker et aux topics.
Il est néanmoins souhaitable que **cet identificateur soit unique** au niveau du broker
...**gare à la recopie multiple de ce code sans comprendre ni lire ! surtout si vous utilisez le même broker.**
- ✓ pas de sécurisation de la connexion (No credential):
Le User Id et son mot de passe ne sont pas fournis : "NULL", "NULL"
C'est aussi possible parce que le serveur/broker le permet. Il pourrait aussi être configuré pour imposer un vrai login.

Cela ne va pas durer ! ;-)

- ⑥ Dans la boucle (Ligne 154), périodiquement on publie (Ligne 174) la payload.

- ⑦ **ET on invoque la fonction "loop" du client MQTT** : Ligne 177

Cette dernière fonction traite les événements MQTT.

Il manque quelques petites choses pour faire le lien avec vos capteurs/actionneurs et notamment la fonction `get_Temperature()` !

3.1.2 Just for information : An example of asynchronous MQTT

L'idée de ce cours est d'éclairer quelques notions et on a déjà fait de l'asynchrone dans le cadre du serveur HTTP.

Pour information, il existe des bibliothèques de clients asynchrones MQTT :

<https://randomnerdtutorials.com/esp32-mqtt-publish-ds18b20-temperature-arduino/>

Mais on ne peut pas tout faire !

3.2 Others MQTT client APIs

3.2.1 With mosquitto CLI

Pour simplifier la mise au point du dialogue, vous pouvez utiliser les commandes "mosquitto" dans un terminal/console pour tester : souscrire/publier sur le topic.

➤ Cela permet de voir facilement la réaction du broker que vous venez d'installer et de configurer.

Dans le contexte qui vient d'être présenté, vous pouvez utiliser ces commandes pour tester le fonctionnement!

- En écoutant le topic de température, et
- En publiant sur le topic led.

3.2.2 With a client in a(ny) programming language (optionnal)

On peut aussi réaliser un client MQTT par la programmation dans un langage quelconque, le choix est large :

<https://cumulocity.com/guides/device-sdk/mqtt-examples/>

Arduino :	https://github.com/256dpi/arduino-mqtt
Processing :	https://github.com/256dpi/processing-mqtt
JavaScript :	https://github.com/mqttjs/MQTT.js
Go :	https://github.com/256dpi/gomqtt
Ruby :	https://github.com/njh/ruby-mqtt
C :	http://www.eclipse.org/paho/clients/c/
C++ :	http://www.eclipse.org/paho/clients/cpp
Java :	http://www.eclipse.org/paho/clients/java
Python :	http://www.eclipse.org/paho/clients/python

3.2.3 With a client in Python

Je vous propose dans ce qui suit un client MQTT écrit en Python (package paho-mqtt anciennement python-mosquitto) et destiné à tourner sur un host/PC permettant ainsi de dialoguer avec l'ESP à la place de la console.

```

1  # Let's talk MQTT in Python
2  # first install : https://pypi.org/project/paho-mqtt/
3  # sudo apt install python3-paho-mqtt sous Ubuntu
4  # Author : G.MENEZ
5  import paho.mqtt.client as mqtt
6  import time
7  import json
8  #-----
9  def on_message(client, userdata, message) : # Callback on arriving messages
10     print("\nmessage received = {}".format(str(message.payload.decode("utf-8"))))
11     print("on topic = {}".format(message.topic))
12     print("with qos = {}".format(message.qos))
13     print("with retain flag = {}".format(message.retain))
14
15  #####
16  if __name__=="__main__":
17     #-----MQTT client instantiation -----
18     clientname = "P1"
19     print("Creating new Client : {}".format(clientname))
20     client = mqtt.Client(clientname) # create new instance
21     client.on_message=on_message # attach function to callback
22
23     #-----connect to broker -----
24     broker_address="test.mosquitto.org" # "192.168.1.101"
25     """ broker_address="iot.eclipse.org" """
26     print("Connecting to broker {}".format(broker_address))
27     client.connect(broker_address) # connect to broker
28
29     client.loop_start() #----- start the loop
30
31     #-----Subscribe -----
32     topicname = "IoT/menez/sensors"
33     print("\nSubscribing to topic {}".format(topicname))
34     client.subscribe(topicname)
35
36     #Make payload .. and Serialize it !
37     payload = {
38         "Temperature":10,
39         "Humidity":50 }
40     msg = json.dumps(payload)
41
42     # Publish it 3 times every 5 sec
43     for i in range(3) :
44         print("\nPublishing message {} to topic {}".format(msg, topicname))
45         client.publish(topicname, payload=msg, qos=2, retain=False)
46         time.sleep(5)
47
48     time.sleep(100) # wait in seconds before
49     client.loop_stop() #----- stop the loop and the script

```

3.2.4 Few comments on this code

Le callback : "on_message"

Vous remarquez la présence de la fonction "on_message" qui est appelée à chaque fois qu'un des topics auxquels on s'est inscrit publie un message.

➤ Ce callback est "attaché" au client MQTT à la ligne 21.

Les boucles ...

From : <https://pypi.org/project/paho-mqtt/#network-loop>

"Network loop"

These functions are the driving force behind the client. If they are not called, incoming network data will not be processed and outgoing network data may not be sent in a timely fashion. There are four options for managing the network loop. Three are described here, the fourth in "External event loop support" below. Do not mix the different loop functions.

3.2.5 With a client in Java (optional work !)

Vous pouvez aussi faire ce client en Java (ou autre) si vous êtes plus confortable dans ce langage :

<http://www.bytesofgigabytes.com/mqtt/java-as-mqtt-publisher-and-subscriber-client/>

4 TODO 1 : La connexion avec Node-Red

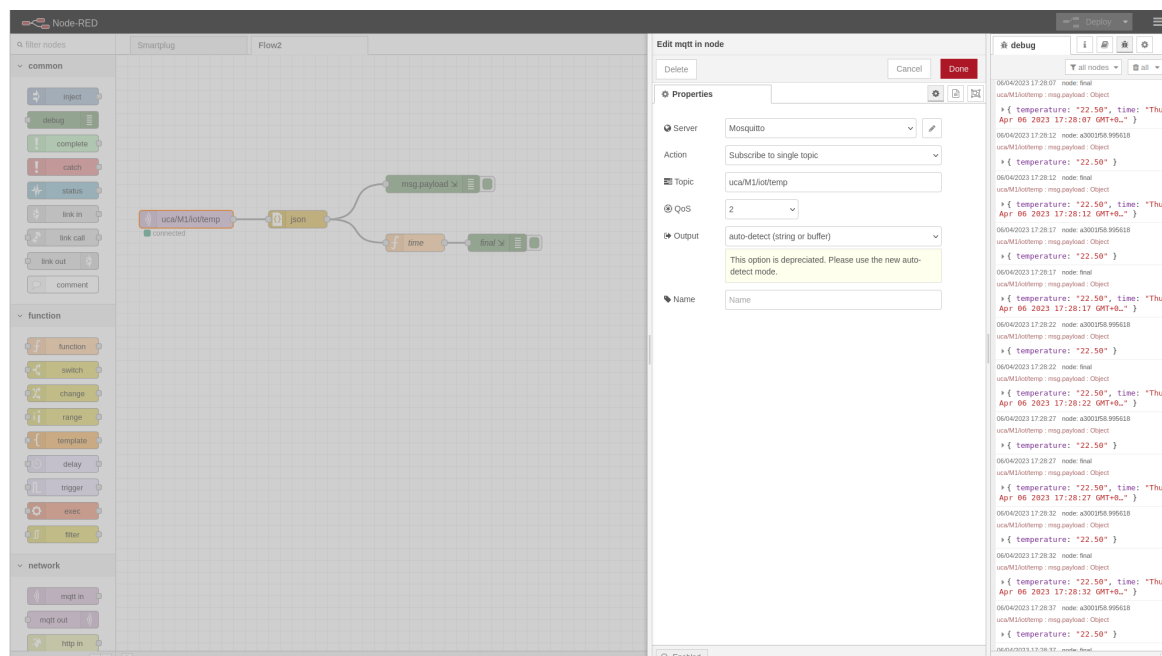
4.1 Au niveau de l'ESP

Pour cette partie, vous modifiez le code de l'ESP pour qu'il publie sur le topic "uca/iot/master" le même JSON qui servait au reporting.

- Vous pourriez faire évoluer le formulaire contenu dans le html stocké sur Little FS pour choisir le broker, le topic et le timing !
- **ATTENTION** à la taille de la payload ... sinon le message ne part pas :-(

4.2 Au niveau de Node Red

Ensuite, on fait en sorte que Node-Red devienne un client MQTT. **Ne soyez pas étonné**, vous allez recevoir les publications de TOUS les objets ... on est en train de former une "flotte" d'objets.



Dans le flot, on a donc placé :

1. un noeud "mqtt_in" (cf palette)
 - Il faut configurer son server/broker ("test.mosquitto.org") et son topic ("uca/iot/master")
Attention aussi à respecter l'unicité de l'identificateur utilisé pour se connecter au broker !
2. un noeud JSON pour convertir la chaîne en "objet" JSON
3. un noeud de debug pour voir que je reçois bien les PUBLISH de l'ESP sur le broker

et on déploie !

On voit bien sur la colonne de droite (le debug) l'arrivée des messages en provenance de l'ESP qui est en train de publier ses températures.

5 Consommation (Optionnel !)

En exploitant les modes présentés dans le document sur la consommation, essayez de restreindre la consommation de l'ESP !